

Отчёт по лабораторной работе №13

Программирование в командном процессоре ОС UNIX. Ветвления и циклы

Терентеевская Светлана Александровна

2026-05-01

Содержание I

- 1 Информация
- 2 Вводная часть
- 3 Выполнение лабораторной работы
- 4 Выводы
- 5 Список литературы

Раздел 1

Информация

- Терентеевская Светлана Александровна

- Терентеевская Светлана Александровна
- Группа НКАбд-05-25, студент бакалавра

- Терентеевская Светлана Александровна
- Группа НКАбд-05-25, студент бакалавра
- Российский университет дружбы народов им. П. Лумумбы

- Терентеевская Светлана Александровна
- Группа НКАбд-05-25, студент бакалавра
- Российский университет дружбы народов им. П. Лумумбы
- 1032253498@rudn.ru

- Терентеевская Светлана Александровна
- Группа НКАбд-05-25, студент бакалавра
- Российский университет дружбы народов им. П. Лумумбы
- 1032253498@rudn.ru
- <https://github.com/SvetlanaTerenteevskaya>

Раздел 2

Вводная часть

Цель работы

- Изучить основы программирования в оболочке ОС UNIX.

Цель работы

- Изучить основы программирования в оболочке ОС UNIX.
- Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Задание

- Реализовать командный файл, анализирующий ключи (-inputfile, -outputfile, -p, -C, -n) с помощью getopt для поиска строк по шаблону в файле.

Задание

- Реализовать командный файл, анализирующий ключи (-inputfile, -outputfile, -p, -C, -n) с помощью getoptс для поиска строк по шаблону в файле.
- Написать программу на С, возвращающей код завершения (знак числа), и командный файл, анализирующий этот код через \$? с выводом соответствующего сообщения.

Задание

- Реализовать командный файл, анализирующий ключи (-inputfile, -outputfile, -p, -C, -n) с помощью getoptс для поиска строк по шаблону в файле.
- Написать программу на С, возвращающей код завершения (знак числа), и командный файл, анализирующий этот код через \$? с выводом соответствующего сообщения.
- Разработать командный файл, создающий N пронумерованных файлов (например, 1.tmp, 2.tmp) или удаляющий их, с передачей N через аргумент командной строки.

Задание

- Реализовать командный файл, анализирующий ключи (-inputfile, -outputfile, -p, -C, -n) с помощью getopt для поиска строк по шаблону в файле.
- Написать программу на C, возвращающей код завершения (знак числа), и командный файл, анализирующий этот код через \$? с выводом соответствующего сообщения.
- Разработать командный файл, создающий N пронумерованных файлов (например, 1.tmp, 2.tmp) или удаляющий их, с передачей N через аргумент командной строки.
- Написать командный файл для архивации (через tar) всех файлов в указанной директории, а затем модифицировать его для упаковки только файлов, изменённых за последнюю неделю (используя find).

- В командных процессорах ОС UNIX (оболочках, таких как bash) программирование позволяет автоматизировать сложные задачи администрирования и обработки данных. В отличие от простых последовательных команд, скрипты используют управляющие конструкции: **ветвления** (if-then-else, case) и **циклы** (for, while, until), что делает их полноценным языком сценариев.

- В командных процессорах ОС UNIX (оболочках, таких как bash) программирование позволяет автоматизировать сложные задачи администрирования и обработки данных. В отличие от простых последовательных команд, скрипты используют управляющие конструкции: **ветвления** (if-then-else, case) и **циклы** (for, while, until), что делает их полноценным языком сценариев.
- Ключевым инструментом для разбора параметров командной строки является встроенная команда getopts. Она позволяет обрабатывать ключи (например, -f, -o) и их аргументы, избавляя программиста от ручного анализа позиционных параметров.

- Особое место занимает **код завершения** команды (\$?). Любая программа (включая написанные на Си) возвращает целое число: 0 — успех, ненулевое значение — ошибка или специальное состояние. Это позволяет организовывать цепочки проверок и условное выполнение.

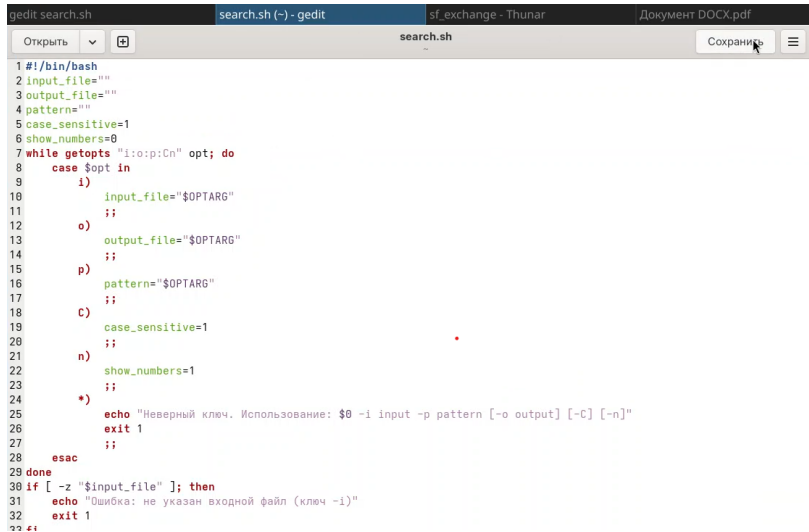
- Особое место занимает **код завершения** команды (\$?). Любая программа (включая написанные на Си) возвращает целое число: 0 — успех, ненулевое значение — ошибка или специальное состояние. Это позволяет организовывать цепочки проверок и условное выполнение.
- Для работы с файлами часто используются генерация имён с помощью **метасимволов** (*, ?, []), а также команды find (фильтрация по времени изменения) и tar (архивация). Понимание различий между циклами while (выполняется, пока условие истинно) и until (выполняется, пока условие ложно) необходимо для корректной реализации повторяющихся действий.

Раздел 3

Выполнение лабораторной работы

Задание 1

Используя команды `getopts` `grep`, я написала командный файл, который анализирует командную строку с ключами: `-iinputfile`, `-ooutputfile`, `-р`шаблон, `-C`, `-n`, а затем ищет в указанном файле нужные строки, определяемые ключом `-р` (рис. 1).

The image shows a screenshot of a gedit text editor window. The title bar at the top contains four tabs: 'gedit search.sh', 'search.sh (~) - gedit', 'sf_exchange - Thunar', and 'Документ DOCX.pdf'. The main window has a menu bar with 'Открыть', a dropdown arrow, and a '+' icon. Below the menu bar, the filename 'search.sh' is displayed. On the right side of the window, there are two buttons: 'Сохранить' (Save) and a hamburger menu icon. The editor area contains a shell script named 'search.sh'. The script starts with a shebang line and sets default values for 'input_file', 'output_file', 'pattern', 'case_sensitive', and 'show_numbers'. It then enters a 'while' loop that processes command-line options: '-i' for input file, '-o' for output file, '-p' for pattern, '-C' for case sensitivity, and '-n' for showing line numbers. An error message is printed for invalid options, and the script exits with a status of 1. Finally, it checks if an input file was specified and prints an error if not. The script ends with a 'done' statement for the while loop and an 'exit' statement.

```
1#!/bin/bash
2input_file=""
3output_file=""
4pattern=""
5case_sensitive=1
6show_numbers=0
7while getopts "i:o:p:Cn" opt; do
8    case $opt in
9        i)
10            input_file="$OPTARG"
11            ;;
12        o)
13            output_file="$OPTARG"
14            ;;
15        p)
16            pattern="$OPTARG"
17            ;;
18        C)
19            case_sensitive=1
20            ;;
21        n)
22            show_numbers=1
23            ;;
24        *)
25            echo "Неверный ключ. Использование: $0 -i input -p pattern [-o output] [-C] [-n]"
26            exit 1
27            ;;
28    esac
29done
30if [ -z "$input_file" ]; then
31    echo "Ошибка: не указан входной файл (ключ -i)"
32    exit 1
33fi
```

Рисунок 1: Файл searh.sh

Создала текстовый файл для проверки программы (рис. 2).



The screenshot shows a gedit text editor window. The title bar at the top displays 'gedit testt.txt', 'testt.txt (~) - gedit', and 'sf_exchange - Tl'. Below the title bar is a toolbar with buttons for 'Открыть' (Open), a dropdown arrow, and a '+' icon. The main text area contains four lines of text, each preceded by a line number: '1 Hello world', '2 HELLO world', '3 hello world', and '4 hello WORLD'. The cursor is positioned at the end of the fourth line.

```
1 Hello world
2 HELLO world
3 hello world
4 hello WORLD
```

Рисунок 2: Текстовый файл

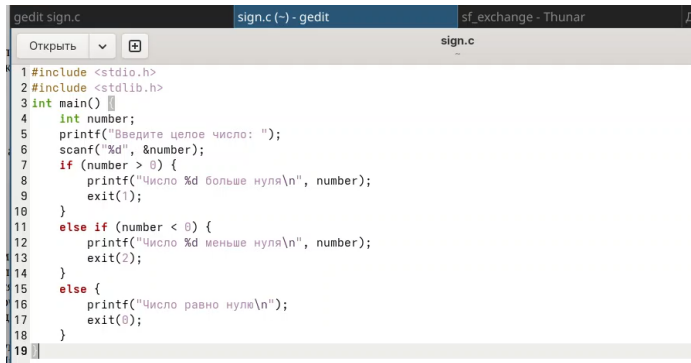
Сделала файл search.sh исполняемым и проверила его работу с разными ключами (рис. 3).

```
saterenteevskaya@saterenteevskaya:~$ chmod +x search.sh
saterenteevskaya@saterenteevskaya:~$ ./search.sh -i testt.txt -p hello
Hello world
HELLO world
hello world
hello WORLD
saterenteevskaya@saterenteevskaya:~$ gedit search.sh
saterenteevskaya@saterenteevskaya:~$ chmod +x search.sh
saterenteevskaya@saterenteevskaya:~$ ./search.sh -i testt.txt -p hello
Hello world
HELLO world
hello world
hello WORLD
saterenteevskaya@saterenteevskaya:~$ ./search.sh -i testt.txt -p hello -C
hello world
hello WORLD
saterenteevskaya@saterenteevskaya:~$ ./search.sh -i testt.txt -p Hello -C
Hello world
saterenteevskaya@saterenteevskaya:~$ ./search.sh -i testt.txt -p Hello -n
1:Hello world
2:HELLO world
3:hello world
4:hello WORLD
saterenteevskaya@saterenteevskaya:~$ |
```

Рисунок 3: Запуск search.sh

Задание 2

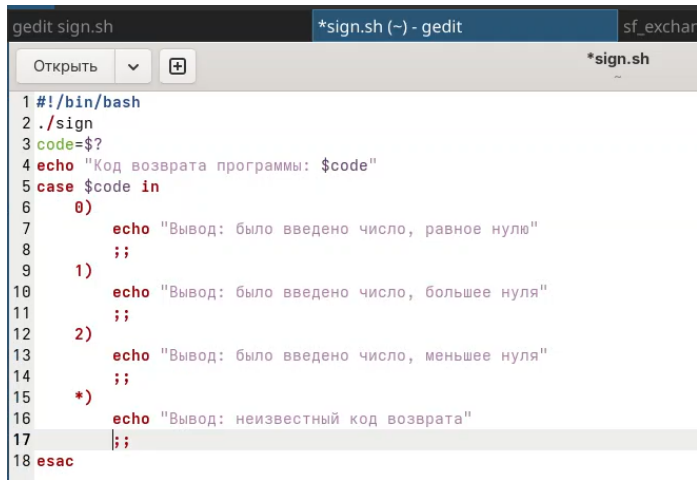
Я написала на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию в о коде завершения в оболочку (рис. 4).



```
gedit sign.c      sign.c (-) - gedit      sf_exchange - Thunar      Д
Открыть  ▾  +      sign.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 int main() {
4     int number;
5     printf("Введите целое число: ");
6     scanf("%d", &number);
7     if (number > 0) {
8         printf("Число %d больше нуля\n", number);
9         exit(1);
10    }
11    else if (number < 0) {
12        printf("Число %d меньше нуля\n", number);
13        exit(2);
14    }
15    else {
16        printf("Число равно нулю\n");
17        exit(0);
18    }
19 }
```

Рисунок 4: Файл sign на Си

Дальше я написала командный файл, который вызывает эту программу и, проанализировав с помощью команды `$?`, выдает сообщение о том, какое число было введено (рис. 5).



```
gedit sign.sh *sign.sh (~) - gedit sf_exchar
Открыть + *sign.sh
1 #!/bin/bash
2 ./sign
3 code=$?
4 echo "Код возврата программы: $code"
5 case $code in
6     0)
7         echo "Вывод: было введено число, равное нулю"
8         ;;
9     1)
10        echo "Вывод: было введено число, большее нуля"
11        ;;
12    2)
13        echo "Вывод: было введено число, меньшее нуля"
14        ;;
15    *)
16        echo "Вывод: неизвестный код возврата"
17    ;;
18 esac
```

Рисунок 5: Командный файл sign.sh

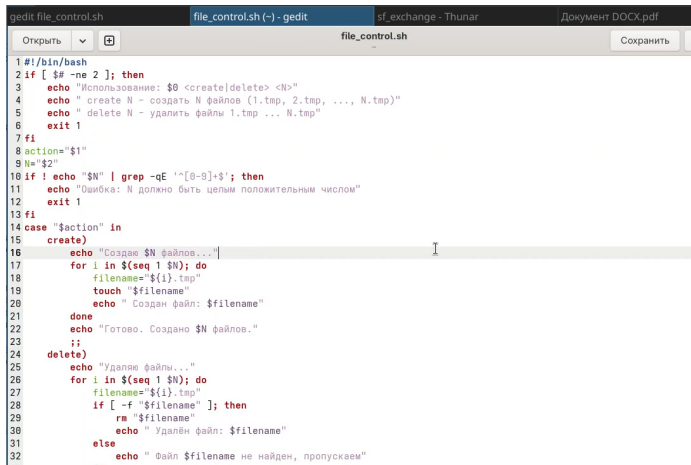
Сделала файл исполняемым и проверила его работу (рис. 6).

```
saterenteevskaya@saterenteevskaya:~$ chmod +x sign.sh
saterenteevskaya@saterenteevskaya:~$ ./sign.sh
Введите целое число: 9
Число 9 больше нуля
Код возврата программы: 1
Вывод: было введено число, большее нуля
saterenteevskaya@saterenteevskaya:~$ ./sign.sh
Введите целое число: -7
Число -7 меньше нуля
Код возврата программы: 2
Вывод: было введено число, меньшее нуля
saterenteevskaya@saterenteevskaya:~$ ./sign.sh
Введите целое число: 0
Число равно нулю
Код возврата программы: 0
Вывод: было введено число, равное нулю
```

Рисунок 6: Запуск sign

Задание 3

Я написала командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до N . Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют) (рис. 7).

The image shows a screenshot of a gedit text editor window. The title bar at the top indicates the file is 'file_control.sh (~) - gedit'. Below the title bar, there are tabs for 'file_control.sh (~) - gedit', 'sf_exchange - Thunar', and 'Документ DOCX.pdf'. The main editing area contains a shell script for file control. The script starts with a shebang line and a usage message. It defines two actions: 'create' and 'delete'. The 'create' action takes a number of files as input and creates them with sequential names. The 'delete' action takes a list of file names and removes them. The script uses 'echo' for output and 'exit' to terminate. The cursor is positioned at the end of line 16, which is the start of the 'create' action's echo statement.

```
1 #!/bin/bash
2 if [ $# -ne 2 ]; then
3     echo "Использование: $0 <create|delete> <N>"
4     echo " create N - создать N файлов (1.tmp, 2.tmp, ..., N.tmp)"
5     echo " delete N - удалить файлы 1.tmp ... N.tmp"
6     exit 1
7 fi
8 action="$1"
9 N="$2"
10 if ! echo "$N" | grep -qE '^[0-9]+$'; then
11     echo "Ошибка: N должно быть целым положительным числом"
12     exit 1
13 fi
14 case "$action" in
15     create)
16         echo "Создаю $N файлов..."
17         for i in $(seq 1 $N); do
18             filename="${i}.tmp"
19             touch "$filename"
20             echo " Создан файл: $filename"
21         done
22         echo "Готово. Создано $N файлов."
23         ;;
24     delete)
25         echo "Удаляю файлы..."
26         for i in $(seq 1 $N); do
27             filename="${i}.tmp"
28             if [ -f "$filename" ]; then
29                 rm "$filename"
30                 echo " Удалён файл: $filename"
31             else
32                 echo " Файл $filename не найден, пропускаем"
```

Рисунок 7: Командный файл file_control.sh

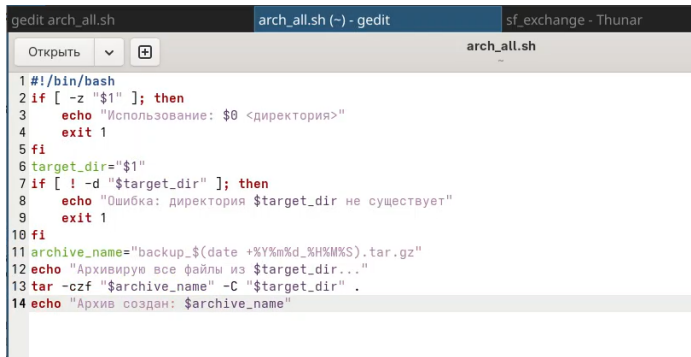
Сделала исполняемым и запустила (рис. 8).

```
saterenteevskaya@saterenteevskaya:~$ chmod +x file_control.sh
saterenteevskaya@saterenteevskaya:~$ ./file_control.sh create 5
bash: ./file_control.sh: Нет такого файла или каталога
saterenteevskaya@saterenteevskaya:~$ ./file_control.sh create 5
Создан 5 файлов...
Создан файл: 1.tmp
Создан файл: 2.tmp
Создан файл: 3.tmp
Создан файл: 4.tmp
Создан файл: 5.tmp
Готово. Создано 5 файлов.
saterenteevskaya@saterenteevskaya:~$ ./file_control.sh delete 3
Удаляю файлы...
Удалён файл: 1.tmp
Удалён файл: 2.tmp
Удалён файл: 3.tmp
Готово.
saterenteevskaya@saterenteevskaya:~$ ls *.tmp
4.tmp 5.tmp
saterenteevskaya@saterenteevskaya:~$ ./file_control.sh delete 2
Удаляю файлы...
Файл 1.tmp не найден, пропускаем
Файл 2.tmp не найден, пропускаем
Готово.
saterenteevskaya@saterenteevskaya:~$ ./file_control.sh delete 5
Удаляю файлы...
Файл 1.tmp не найден, пропускаем
Файл 2.tmp не найден, пропускаем
Файл 3.tmp не найден, пропускаем
Удалён файл: 4.tmp
Удалён файл: 5.tmp
Готово.
saterenteevskaya@saterenteevskaya:~$ |
```

Рисунок 8: Запуск file_control.sh

Задание 4

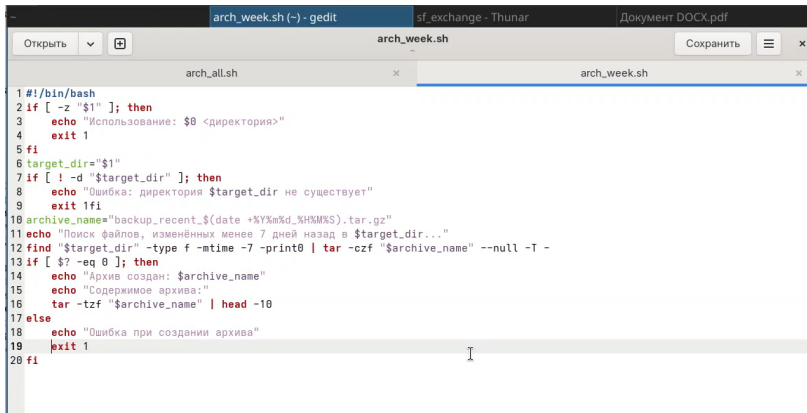
Написала командный файл, который с помощью команды `tar` запаковывает в архив все файлы в указанной директории (рис. 9).



```
gedit arch_all.sh  arch_all.sh (~) - gedit  sf_exchange - Thunar
Открыть  arch_all.sh
1 #!/bin/bash
2 if [ -z "$1" ]; then
3     echo "Использование: $0 <директория>"
4     exit 1
5 fi
6 target_dir="$1"
7 if [ ! -d "$target_dir" ]; then
8     echo "Ошибка: директория $target_dir не существует"
9     exit 1
10 fi
11 archive_name="backup_$(date +%Y%m%d_%H%M%S).tar.gz"
12 echo "Архивирую все файлы из $target_dir..."
13 tar -czf "$archive_name" -C "$target_dir" .
14 echo "Архив создан: $archive_name"
```

Рисунок 9: Файл `arch_all.sh`

Далее модифицировала его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовала команду `find`), создав для этого второй файл (рис. 10).



```
1 #!/bin/bash
2 if [ -z "$1" ]; then
3     echo "Использование: $0 <директория>"
4     exit 1
5 fi
6 target_dir="$1"
7 if [ ! -d "$target_dir" ]; then
8     echo "Ошибка: директория $target_dir не существует"
9     exit 1
10 archive_name="backup_recent_$(date +%Y%m%d_%H%M%S).tar.gz"
11 echo "Поиск файлов, изменённых менее 7 дней назад в $target_dir..."
12 find "$target_dir" -type f -mtime -7 -print0 | tar -czf "$archive_name" --null -T -
13 if [ $? -eq 0 ]; then
14     echo "Архив создан: $archive_name"
15     echo "Содержимое архива:"
16     tar -tzf "$archive_name" | head -10
17 else
18     echo "Ошибка при создании архива"
19     exit 1
20 fi
```

Рисунок 10: Файл `arch_week.sh`

Сделала файлы исполняемыми, создала тестовую директорию с файлами: для одного изменила дату, а другой не трогала и запустила (рис. 11).

```
saterenteevskaya@saterenteevskaya:~$  
saterenteevskaya@saterenteevskaya:~$ chmod +x arch_week.sh  
saterenteevskaya@saterenteevskaya:~$ chmod +x arch_all.sh  
saterenteevskaya@saterenteevskaya:~$ mkdir -p test_dir  
saterenteevskaya@saterenteevskaya:~$ cd test_dir  
saterenteevskaya@saterenteevskaya:~/test_dir$ echo "Старое содержимое" > old_file.txt  
saterenteevskaya@saterenteevskaya:~/test_dir$ touch -t 202001091120 old_file.txt  
saterenteevskaya@saterenteevskaya:~/test_dir$ echo "Новое содержимое" > new_file.txt  
saterenteevskaya@saterenteevskaya:~/test_dir$ ls  
new_file.txt  old_file.txt  
saterenteevskaya@saterenteevskaya:~/test_dir$ cd  
saterenteevskaya@saterenteevskaya:~$ ./arch_all.sh test_dir  
Архивирую все файлы из test_dir...  
Архив создан: backup_20260412_002402.tar.gz  
saterenteevskaya@saterenteevskaya:~$ ./arch_week.sh test_dir  
./arch_week.sh: строка 21: syntax error: unexpected end of file from `if' command on line 7  
saterenteevskaya@saterenteevskaya:~$ gedit arch_week.sh  
(gedit:18643): Gtk-WARNING **: 00:26:51.586: Calling org.freedesktop.portal.Inhibit.Inhibit failed: GDBus.Error:org.freedesktop.DBus.Error.UnknownMethod: Не существует интерфейса "org.freedesktop.portal.Inhibit" на объекте по пути /org/freedesktop/portal/desktop  
saterenteevskaya@saterenteevskaya:~$ chmod +x arch_week.sh  
saterenteevskaya@saterenteevskaya:~$ ./arch_week.sh test_dir  
Поиск файлов, изменённых менее 7 дней назад в test_dir...  
Архив создан: backup_recent_20260412_002723.tar.gz  
Содержимое архива:  
test_dir/new_file.txt  
saterenteevskaya@saterenteevskaya:~$ |
```

Рисунок 11: Запуск arch_all и arch_week

Раздел 4

Выводы

В ходе выполнения данной лабораторной работы я изучила основы программирования в оболочке ОС UNIX и научилась писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Раздел 5

Список литературы

ТУИС
Методичка к лабораторной работе №13